
TRACE

Trace-based Runtime Automata for Compliance and Enforcement

System and Method for Semantic Abstraction, Probabilistic
Automata-Based Behavioral Protocol Inference, and Runtime
Conformance Verification of Heterogeneous AI-Agent Execution Traces

Version: TRACE v2.0

Date: September 25, 2026

Status: Implemented research-grade prototype

Contents

1	Title of the Invention	3
2	Field / Area of the Invention	3
3	Prior Patents and Publications from Literature	4
3.1	Patents	4
3.2	Research Publications	7
3.3	Prior-Art Feature Comparison Matrix	10
4	Summary and Background of the Invention	12
4.1	Background	12
4.2	Problem Statement	12
4.3	Existing Gap	12
4.4	Implemented Solution	13
5	Objectives of the Invention	13
6	Working Principle of the Invention	14
6.1	End-to-End Workflow	14
6.2	Workflow / Block Diagram	14
6.3	Runtime Operation (Online Path)	14
6.4	Learning Operation (Offline Path)	15
7	Detailed Description of the Invention	15
7.1	System Architecture (Modules M1–M12)	15
7.2	Canonical Event Schema (CES)	17
7.3	Secret Redaction	17
7.4	Framework Adapters	17
7.5	Semantic Abstraction	18
7.6	Protocol Inference	18
7.7	Hierarchical Trace Folding	20
7.8	Runtime Verification and the Product Automaton	21
7.9	Counterexample Generation	22
7.10	Drift Detection	22
7.11	Computational Complexity	23
7.12	Security and Failure Modes	23
7.13	End-to-End Operational Swimlane	25
7.14	Operational Interfaces and Deployment	27
7.15	Recommended Patent Drawing Set	27
8	Experimental Validation Results	28
8.1	Current Implementation Status	28
8.2	Automated Testing Evidence	28
8.3	Empirical Research-Question Results	29
8.4	RQ1: Sample Complexity	30
8.5	RQ2: Automata Recovery	30
8.6	RQ3: Verification Latency	30
8.7	RQ4: Policy Enforcement	30
8.8	RQ5: Drift Detection	31
8.9	RQ6 Status: Hierarchical State Space Reduction (Analytical Estimate)	31
8.10	Semantic Abstraction Ablation	31
8.11	Interpretation of the Current Evidence	32

8.12 Evidence Not Currently Supplied	32
9 Aspects of the Invention Needing Protection	32
9.1 Potentially Distinguishing System Architecture	32
9.2 Potentially Distinguishing Method	33
9.3 Potentially Distinguishing Data Representation	33
9.4 Potentially Distinguishing Anomaly-Detection Mechanism	34
9.5 Potentially Distinguishing Hierarchical Mechanism	34
9.6 Potentially Distinguishing Drift / Lifecycle Mechanism	34
9.7 Potentially Distinguishing Feedback Mechanism	34
9.8 Independent Claim Concept	35
9.9 Possible Dependent-Claim Concepts	35
9.10 Novelty versus Obviousness	35
10 Technology Readiness Level (TRL)	36
Executive Novelty Assessment	38
Critical Issues Requiring Inventor Review	39

1. Title of the Invention

Recommended Title

“System and Method for Semantic Abstraction, Probabilistic Automata-Based Behavioral Protocol Inference, and Runtime Conformance Verification of Heterogeneous AI-Agent Execution Traces”

Alternative Titles

- (a) “Trace-Based Behavioral Protocol Inference and Hierarchical Runtime Conformance Checking for Multi-Framework AI Agent Systems”
- (b) “Method and System for Inferring Probabilistic Finite-State Behavioral Models from Heterogeneous Agent Execution Logs and Detecting Runtime Policy Deviations”

The recommended title deliberately identifies the principal technical elements of the system-level combination: semantic abstraction of heterogeneous traces, probabilistic automata inference, and runtime conformance verification. The title does not itself assert that any particular combination satisfies a legal novelty or inventive-step standard.

2. Field / Area of the Invention

The invention lies at the intersection of several established technical fields:

- **Artificial intelligence / autonomous agent systems** — systems in which a large-language-model-driven planner selects and sequences tool invocations, sub-agent delegations, and memory operations at runtime.
- **Software observability and execution-trace analysis** — capture and structuring of runtime event streams from distributed, multi-component software systems.
- **Automata theory and grammatical inference** — passive learning of finite-state and probabilistic finite-state representations from example sequences, including state-merging approaches such as RPNI, EDSM, and ALERGIA/MDI.
- **Runtime verification / software model checking** — checking an observed execution against a formal specification, as distinct from exhaustive state-space model checking of all possible executions.
- **Process mining** — discovery of process models from event logs and conformance checking of new traces against discovered or specified models.
- **Anomaly and intrusion detection** — behavior-based detection of deviations from a learned baseline of normal system activity.
- **Machine learning** — semantic representations and clustering techniques that may be used to normalize action vocabularies across independently developed software frameworks.
- **Cybersecurity / AI safety** — policy enforcement and safe operation of autonomous or semi-autonomous software agents that take real-world actions.

The present system specifically applies these areas to multi-framework AI-agent execution traces. Such traces can exhibit stochastic action selection, heterogeneous action vocabularies, nested delegation, incomplete observations, and behavioral changes over time.

The implementation evidence described later in this disclosure establishes a working software prototype and controlled benchmark evaluation. It does not establish production deployment or universal performance across all agent architectures, hardware, datasets, or workloads.

3. Prior Patents and Publications from Literature

The prior-art discussion below is retained from the supplied disclosure and is presented as a technical comparison rather than a legal patentability opinion. No new prior-art search is asserted in this revision. The references and publication details are retained from the supplied material.

3.1 Patents

Table 1: Comparative summary of directly relevant patents.

Sl. No.	Patent No. / Publication	Year	Patent Office / Assignee	Key Contribution	Limitation (relative to TRACE)	Technical Distinction / Proposed Contribution
1	US 11,989,296 B2 — “Program Execution Anomaly Detection for Cybersecurity”	2024 (priority 2022)	USPTO; CybersentryAi, Inc.	Instruments a supervised program in a model-building mode, trains an acceptable-behavior model from event sequences, and compares live event sequences against it in an operation mode; uses preceding event context as model input.	Operates on a single instrumented program/process; the supplied comparison does not identify multi-framework symbolic normalization, semantic action abstraction, probabilistic state-merging inference, bounded delegation modeling, or explicit product composition with a separately authored policy automaton.	TRACE combines heterogeneous agent-trace normalization, probabilistic state-merging inference, bounded hierarchical delegation, and independent learned/policy verification tracks in one architecture.
2	US 2018/0129579 A1 — “Systems and Methods with a Realtime Log Analysis Framework” (LogLens)	2018 (priority 2016)	USPTO; NEC Laboratories America, Inc.	Unsupervised event identification over heterogeneous multi-source logs; constructs event automata and performs real-time sequence-anomaly detection with dynamic model updates.	The described automata are frequency/association-based state machines over log fields, rather than the TRACE PDFA state-merging representation. The reference does not establish TRACE’s agent-framework normalization, bounded delegation treatment, or separate policy-automaton product track.	TRACE uses a probabilistic state-merging representation and adds canonicalization across agent frameworks, hierarchical delegation handling, and independent explicit-policy verification.
3	EP 1063828 B1 / US 7,369,507 B1 — “Method for Identifying the Underlying Finite State Machines of a Protocol Implementation”	2006 (priority 1999)	EPO / USPTO; Tektronix Inc. (later NetScout)	Learns finite-state representations underlying protocol implementations from example traces by clustering observed time points into equivalence classes.	Targets classical communications protocols and does not establish the combination of stochastic PDFA inference, heterogeneous AI-agent semantic abstraction, bounded delegation, or separate policy-product verification used by TRACE.	TRACE applies probabilistic automata inference to AI-agent execution behavior and combines it with semantic cross-framework normalization, bounded hierarchical composition, and explicit policy verification.

(Table 1 continued)

Sl. No.	Patent No. / Publication	Year	Patent Office / Assignee	Key Contribution	Limitation (relative to TRACE)	Technical Distinction / Proposed Contribution
4	US 11,928,629 B2 — “Graph Encoders for Business Process Anomaly Detection”	2023 (filed approximately 2022)	USPTO; International Business Machines Corporation	Converts business-process event logs into graph structures, learns graph encodings using unsupervised ML, and computes process-aware anomaly scores.	The representation is graph-embedding based rather than an explicit probabilistic finite-state behavioral model. The supplied reference does not establish TRACE’s hierarchical delegation representation, learned/policy product construction, or cross-framework agent-trace normalization.	TRACE retains explicit finite-state transition structure and probabilistic transition semantics while adding semantic abstraction, bounded delegation, policy composition, and drift monitoring.
5	US 9,736,172 B2 — “Signature-Free Intrusion Detection”	2017 (priority 2007)	USPTO; Avaya Inc. (chain of successor assignees)	Maintains finite-state models for sessions/nodes/protocol combinations and flags incompatible states or transitions; composes local and global constraints.	The supplied comparison describes hand-specified finite-state models rather than passive probabilistic state-merging learning from positive execution traces. It does not establish semantic cross-framework action abstraction, bounded agent delegation, or drift-driven model lifecycle management.	TRACE infers probabilistic behavioral models from execution traces and independently composes them with an explicit policy DFA, while addressing heterogeneous AI-agent execution and bounded delegation.

3.2 Research Publications

Table 2: Comparative summary of directly relevant research publications.

Sl. No.	Publication	Year	Venue	Key Contribution	Limitation (relative to TRACE)	Technical Distinction / Proposed Contribution
1	J. Oncina and P. Garc'ia, "Inferring Regular Languages in Polynomial Updated Time"	1992	Pattern Recognition and Image Analysis, World Scientific	Introduces RPNI, a foundational state-merging approach for learning canonical DFAs from example strings.	Targets regular-language learning and does not provide probabilistic semantics, cross-framework AI-agent normalization, hierarchical delegation, or policy-product verification.	TRACE does not claim RPNI as inventive. The candidate contribution lies in the surrounding system that normalizes agent traces, learns probabilistic behavioral models, represents bounded delegation, and performs joint learned/policy verification.
2	S. Verwer and C.,A. Hammerschmidt, "FlexFringe: A Passive Automaton Learning Package" (ICSME 2017); extended as "FlexFringe: Modeling Software Behavior by Learning Probabilistic Automata"	2017 / 2022	IEEE ICSME 2017 / arXiv	Provides EDSM/ALERGIA-style state-merging machinery for probabilistic automaton learning from software logs.	It is the underlying inference technology used by TRACE and does not by itself establish TRACE's cross-framework semantic normalization, bounded delegation composition, separate policy DFA, or implemented HITL/drift lifecycle.	TRACE is explicitly built around established automata-learning machinery. The candidate system-level distinction is the orchestration and interaction of that machinery with the agent-trace normalization, hierarchy, policy, verification, drift, and feedback layers.
3	W.,M.,P. van der Aalst and A.,K. Alves de Medeiros, "Process Mining and Security: Detecting Anomalous Process Executions and Checking Process Conformance"	2005	Electronic Notes in Theoretical Computer Science, vol. 121	Establishes discovering process models from event logs and checking executions for conformance in a security/anomaly-detection setting.	Targets classical process logs and does not establish the TRACE PDFa likelihood semantics, agent-framework normalization, semantic action abstraction, bounded delegation structure, or explicit learned/policy product automaton.	TRACE uses explicit probabilistic automata and likelihood-based conformance while targeting heterogeneous, stochastic AI-agent execution traces.

(Table 2 continued)

Sl. No.	Publication	Year	Venue	Key Contribution	Limitation (relative to TRACE)	Technical Distinction / Proposed Contribution
4	M. Leemans, W.,M.,P. van der Aalst, and M.,G.,J. van den Brand, “Recursion Aware Modeling and Discovery for Hierarchical Software Event Log Analysis”	2018	IEEE SANER 2018	Introduces recursion-aware process-model discovery leveraging hierarchical call information in software event logs.	Targets software event-log hierarchy and process-tree discovery rather than probabilistic automaton inference for multi-agent systems.	TRACE uses bounded role-specific child automata associated with delegate(role) events within a probabilistic runtime-verification architecture.
5	J. Liu, B. Ruan, X. Yang, Z. Lin, Y. Liu, Y. Wang, T. Wei, and Z. Liang, “TraceAegis: Securing LLM-Based Agents via Hierarchical and Behavioral Anomaly Detection”	2025	arXiv preprint (cs.CR)	Provides a provenance-based framework for hierarchical behavioral anomaly detection in LLM-agent execution traces and derives behavioral constraints for validation.	The supplied comparison identifies differences in formal PDFA state-merging inference, separate policy-product composition, cross-framework semantic normalization, and drift-triggered model lifecycle. These differences require careful attorney review rather than being treated as legally dispositive.	TraceAegis is the closest identified reference in the supplied review. TRACE’s candidate technical distinctions are the particular interaction between probabilistic automata, cross-framework normalization, bounded delegation, explicit policy composition, drift monitoring, and HITL lifecycle.

3.3 Prior-Art Feature Comparison Matrix

Table 3: Condensed feature-level comparison against the supplied prior-art set.

Technical Feature	US 11,989,296	US 2018/0129579	EP 1063828	US 11,928,629	US 9,736,172	TraceAegis (2025)	TRACE
Heterogeneous multi-framework agent adapters	No	No	No	No	No	Partial	Yes
Canonical event schema / cross-framework join point	No	Partial	No	Partial	No	Partial	Yes
Semantic action abstraction	No	No	No	No	No	Unclear	Implemented mechanism; validation evidence limited
Probabilistic automata learning	No	Partial	Partial	No	No	No	Yes
Hierarchical delegation modeling	No	No	No	No	No	Yes	Yes
Runtime conformance checking	Yes	Yes	Yes	Partial	Yes	Yes	Yes
Explicit policy automaton separate from learned model	No	No	No	No	Partial	No	Yes
Product automaton learned-state $\times$ policy-state	No	No	No	No	Partial	No	Yes
Behavioral drift detection	No	Partial	No	No	No	No	Yes
Counterexample generation	No	Partial	No	Partial	No	Partial	Yes
Human-in-the-loop feedback	No	Partial	No	No	No	No	Yes
Windowed / drift-triggered relearning	No	Yes	No	No	No	Unclear	Implemented lifecycle mechanism

Technical comparison of the closest combination. The supplied review did not identify a single reference disclosing the complete TRACE combination. The closest conceptual comparison is **TraceAegis (2025)** for LLM-agent behavioral security, considered alongside established automata-learning references and the identified patents concerning behavioral anomaly detection, runtime monitoring, and finite-state composition.

This observation is a technical comparison only. It does not establish legal novelty, non-obviousness, validity, or patentability. In particular, whether the specific combination would have been obvious to a person skilled in the relevant art requires formal patent-law analysis.

4. Summary and Background of the Invention

4.1 Background

Modern AI-agent frameworks such as Model Context Protocol (MCP), LangGraph, CrewAI, AutoGen/AG2, OpenAI Agents SDK, Semantic Kernel, and Google ADK permit LLM-driven systems to select and sequence tool invocations, observations, retries, memory operations, and delegated subtasks at runtime.

TRACE addresses the resulting observability and verification problem by treating agent execution traces as symbolic behavioral sequences rather than as unstructured log records. The implemented prototype accepts traces through seven agent-framework adapters and two benchmark-trajectory adapters, normalizes them into a Canonical Event Schema, learns probabilistic behavioral models, and verifies observed executions against both learned behavior and explicit policy.

4.2 Problem Statement

AI-agent execution traces exhibit several properties that complicate conventional monitoring:

- (a) stochastic action selection arising from LLM-based planning;
- (b) heterogeneous action vocabularies across independently developed frameworks;
- (c) nested structure caused by sub-agent delegation;
- (d) behavioral change as prompts, models, tools, and agent configurations evolve;
- (e) incomplete availability of manually authored specifications for all legitimate trajectories.

A useful runtime mechanism therefore benefits from distinguishing at least three conditions:

1. a transition structurally absent from the learned behavioral model;
2. a transition present in the learned model but sufficiently unlikely;
3. a transition prohibited by an explicit safety or compliance policy.

These conditions have different semantics and can require different operational responses.

4.3 Existing Gap

The individual technologies used by TRACE are established. These include passive automata learning, probabilistic automata, state merging, semantic representations, runtime verification, process conformance, hierarchical execution modeling, anomaly detection, and finite-state product construction.

The candidate technical distinction is therefore not any one of these components in isolation. It concerns their particular interaction when applied to heterogeneous multi-framework AI-agent traces.

The supplied implementation and benchmark evidence demonstrates that this architecture has progressed beyond a concept-only design. Empirical benchmark measurements are established for RQ1–RQ5 alongside an empirical vocabulary compression ablation (64.29%) and an analytical state-space reduction model for RQ6 (70.83%), while empirical multi-agent workload traces and embedding cluster purity remain future evaluation targets.

4.4 Implemented Solution

TRACE v2.0 implements a pipeline that:

1. receives execution events from heterogeneous agent frameworks and benchmark trajectory sources;
2. normalizes valid events into Canonical Event Schema (CES) v1.0 records;
3. performs recognized secret redaction before parameter-schema hashing;
4. stores normalized events in SQLite;
5. constructs symbolic traces and a Prefix Tree Acceptor;
6. performs state-merging inference using ALERGIA/MDI-style statistical compatibility and EDSM;
7. represents the resulting learned behavior as a probabilistic deterministic finite automaton;
8. represents delegated execution using bounded-depth role-specific trace corpora and `delegate(role)` boundaries;
9. compiles explicit policy rules into a policy DFA;
10. jointly tracks learned and policy states during runtime verification;
11. distinguishes policy violations, structural anomalies, and statistical anomalies;
12. generates counterexample explanations;
13. monitors rolling conformance distributions using a two-sample KS test and a tail-quantile CUSUM mechanism;
14. supports HITL feedback and a candidate lifecycle of `CANDIDATE` $\rightarrow$ `SHADOWED` $\rightarrow$ `ACTIVE` $\rightarrow$ `SUPERSEDED`;
15. exposes FastAPI, dashboard, Prometheus, CLI, and Docker interfaces; and
16. provides an empirical benchmark harness with reported results for RQ1–RQ5.

No individual component is asserted as newly invented. The candidate technical contribution is the particular system-level combination and the interactions between its components.

5. Objectives of the Invention

1. Normalize heterogeneous, multi-framework AI-agent execution logs into a common Canonical Event Schema without requiring the downstream inference and verification pipeline to understand each source-specific representation.
2. Reduce framework-specific action vocabulary into canonical symbolic representations where the implemented abstraction mechanism supports such normalization, while avoiding an assertion that the supplied evidence establishes a universal compression or accuracy improvement.

3. Infer probabilistic behavioral models from positive execution traces using state-merging techniques suitable for stochastic behavior.
4. Represent recursive multi-agent delegation using bounded-depth hierarchical composition rather than claiming support for unbounded pushdown-style recursion.
5. Distinguish statistically rare behavior from structurally unseen behavior.
6. Independently evaluate learned behavioral conformance and explicit policy conformance using a product-state representation.
7. Detect behavioral distribution shifts using rolling conformance measurements and implemented statistical monitoring mechanisms.
8. Generate developer-facing explanations of detected deviations using canonical action identities.
9. Provide an operational model lifecycle in which candidate models can be shadowed, activated, and superseded with human feedback.

6. Working Principle of the Invention

6.1 End-to-End Workflow

Raw framework or benchmark trace → adapter layer (M1) → Canonical Event Schema → secret redaction and normalization → symbolic trace representation (M2) → trace corpus (M3) → PTA and state-merging inference (M4) → versioned PDFA model repository (M5) → runtime verifier (M6) ← explicit policy DFA (M8) → product-state verification → structural/statistical/policy classification → counterexample explanation (M9) → HITL feedback (M12) and drift monitoring (M7).

The current implementation also exposes API, dashboard, telemetry, CLI, and Docker interfaces.

6.2 Workflow / Block Diagram

6.3 Runtime Operation (Online Path)

For each incoming event, the applicable adapter converts source-specific data into the TRACE event representation. The event is validated against CES v1.0. Recognized secrets are redacted using entropy- and pattern-based detection before parameter-schema hashing.

The normalized event is then associated with the learned symbolic representation used by the current behavioral model. The runtime verifier advances the learned behavioral state and explicit policy state independently while retaining the joint state

$$(q_A, q_P) \in Q_A \times Q_P.$$

The primary classification precedence is:

1. explicit policy violation;
2. structurally unseen transition;
3. low-probability transition;
4. otherwise conformant.

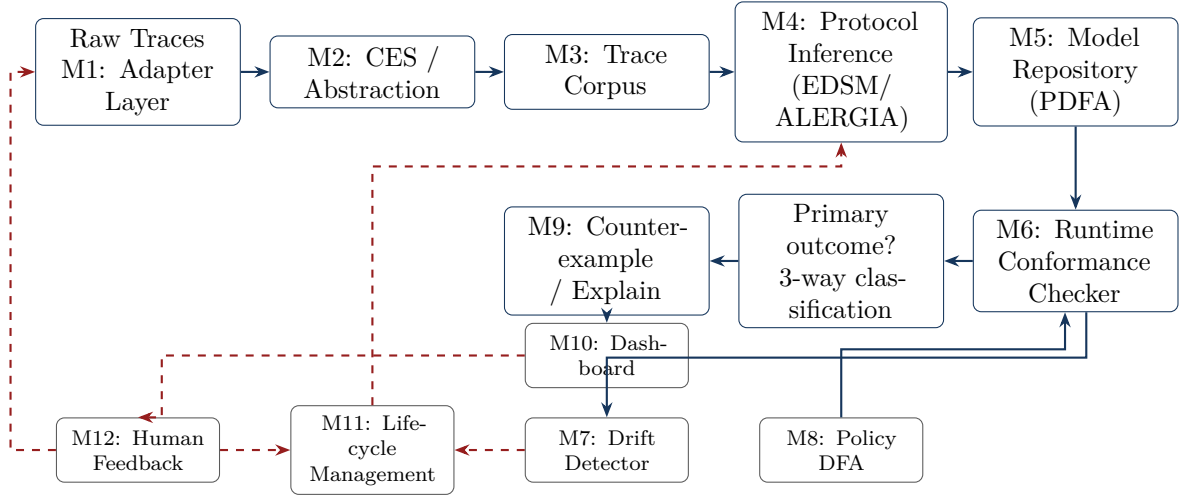


Figure 1: Overall TRACE v2.0 implementation pipeline. Solid blue arrows represent primary data and verification flow. Dashed red arrows represent feedback and model lifecycle interactions. M1 contains seven heterogeneous agent-framework adapters and two benchmark-trajectory adapters. The implementation includes CES validation, state-merging inference, bounded hierarchical delegation, policy-product verification, drift monitoring, HITL feedback, API/dashboard interfaces, and model lifecycle handling.

Where multiple conditions are simultaneously present, secondary diagnostic flags may be retained, but the primary classification follows the above precedence.

This separation ensures that a learned model’s statistical normality cannot override an explicit policy restriction.

6.4 Learning Operation (Offline Path)

Canonical symbolic traces are represented as a Prefix Tree Acceptor (PTA). State and transition evidence is then used by the implemented state-merging inference machinery, including EDSM ordering and ALERGIA/MDI-style statistical compatibility.

The resulting representation is a probabilistic deterministic finite automaton. The current documentation does not establish a Hopcroft-style DFA minimization step for this learned PDFA; therefore no such minimization is asserted here.

The model is stored in a versioned repository and can participate in the implemented candidate lifecycle.

7. Detailed Description of the Invention

7.1 System Architecture (Modules M1–M12)

TRACE is organized into twelve functional modules. Several modules operate as cross-cutting services supporting the online verification and model-lifecycle paths.

Table 4: Module-level architecture summary.

Module	Purpose	Implemented Mechanism / Notes
M1	Trace Ingestion	Nine active trace adapters comprising seven agent-framework adapters (MCP, LangGraph, CrewAI, AutoGen/AG2, OpenAI Agents SDK, Semantic Kernel, Google ADK/Vertex AI) and two benchmark-trajectory adapters (SWE-bench and OSWorld).
M2	CES Normalization and Semantic Abstraction	Canonical Event Schema validation and normalization, including the implemented secret-redaction mechanism. The repository also defines the semantic-symbol abstraction architecture; the supplied README does not provide a quantitative ablation establishing its performance benefit.
M3	Trace Store / Corpus Management	SQLite CES relational persistence, trace corpus management, role-specific corpora, windowing, and trace completeness handling documented by the implementation.
M4	PTA and Protocol Inference	Prefix Tree Acceptor construction, ALERGIA/MDI-style statistical state merging, and EDSM inference machinery.
M5	Versioned PDFA Model Repository	Storage and lifecycle management for learned probabilistic models, including candidate, shadowed, active, and superseded states. No Hopcroft-style PDFA minimization is asserted.
M6	Runtime Verification / Product Automaton	Joint tracking of learned PDFA and explicit policy DFA states with structural, statistical, and policy classification.
M7	Behavioral Drift Monitoring	Rolling conformance/NLL distributions, two-sample KS testing, and tail-quantile CUSUM monitoring.
M8	Policy DSL and Policy DFA Compilation	Declarative policy language and bounded-counter DFA construction for explicit safety and compliance constraints.
M9	Counterexample Generation	Generation of human-readable violation explanations and expected-versus-observed transition differences.
M10	Dashboard / Developer Interface	Implemented web dashboard at <code>/dashboard</code> , including automata visualization, verification interaction, HITL triage, and benchmark presentation elements documented by the repository.
M11	Lifecycle / Relearning Management	Supports model lifecycle operations and drift-related relearning workflow where documented by the implementation. The supplied evidence does not establish a universal automatic scheduling policy for every deployment environment.

Module	Purpose	Implemented Mechanism / Notes
M12	Human-in-the-Loop Feedback / Triage	Violation triage and candidate model lifecycle support, including feedback recording and promotion/review mechanisms.

7.2 Canonical Event Schema (CES)

TRACE implements Canonical Event Schema (CES) v1.0 with JSON Schema draft 2020-12 validation. The schema provides a stable representation through which events originating from structurally different sources can enter the common processing pipeline.

The documented event representation includes fields associated with agent identity, event type, symbolic action identity, attributes, timestamps, parent-span relationships, and delegation depth.

The implementation separates event-level categorization from the fine-grained symbolic representation used by the inference engine. The supplied repository establishes the CES implementation and validation mechanism, but the README does not provide a complete quantitative evaluation of the semantic abstraction layer.

7.3 Secret Redaction

TRACE implements a secret-redaction mechanism before parameter-schema hashing.

The mechanism combines:

- entropy-based detection; and
- pattern/token matching.

The documented target classes include API keys, Bearer tokens, private keys, and authorization headers.

The mechanism should be understood as a mitigation intended to prevent recognized secrets from entering normalized or hashable parameter representations. The implementation name “Zero-Leak Secret Redactor” does not imply a mathematical or universal guarantee of zero information leakage.

Malformed events failing schema validation can be partitioned into a dead-letter handling path rather than being admitted into the valid CES store.

7.4 Framework Adapters

The current implementation documents nine active trace adapters:

1. Model Context Protocol (MCP);
2. LangGraph;
3. CrewAI;
4. AutoGen / AG2;
5. OpenAI Agents SDK;
6. Semantic Kernel;
7. Google ADK / Vertex AI;
8. SWE-bench; and
9. OSWorld.

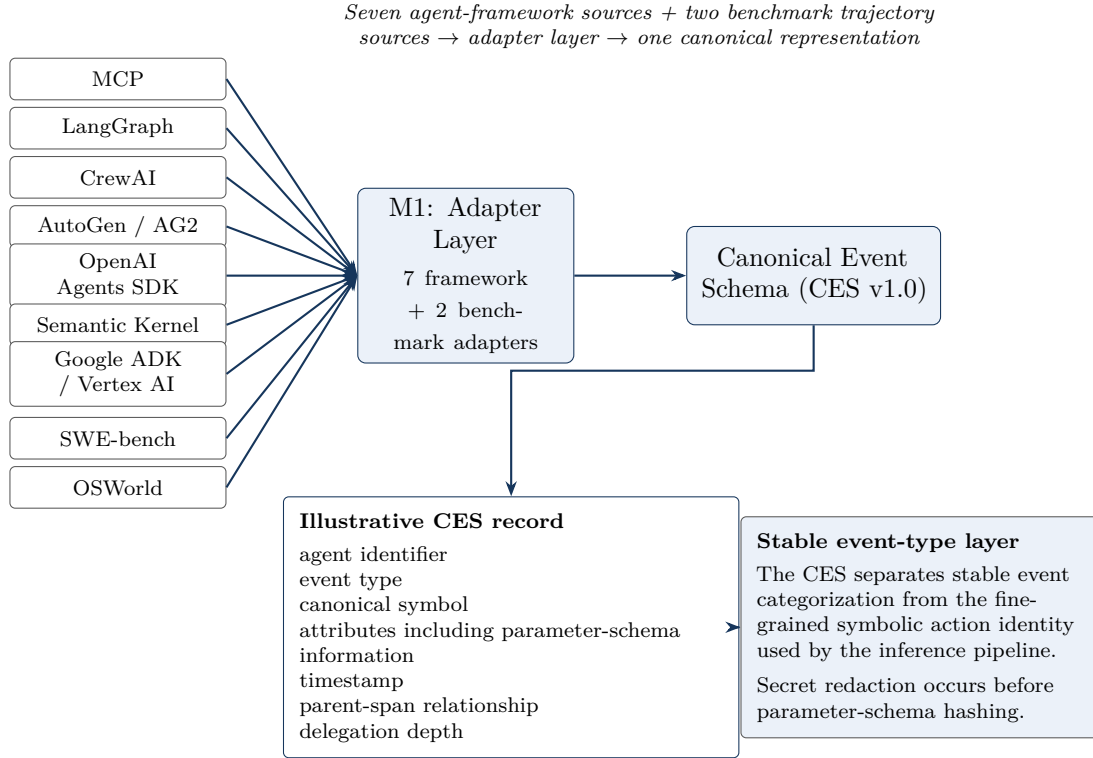


Figure 2: Implemented framework adapter and Canonical Event Schema layer. The current repository documents seven heterogeneous agent-framework adapters and two benchmark trajectory adapters. Valid events enter the CES pipeline after schema validation and recognized secret redaction; malformed schema records are handled through the dead-letter path.

The first seven are heterogeneous agent-framework adapters. SWE-bench and OSWorld are benchmark-trajectory adapters and are not themselves agent frameworks.

This distinction is important because benchmark trajectory importers and live framework adapters have different source semantics and should not be represented as nine equivalent framework integrations.

7.5 Semantic Abstraction

The architecture describes semantic abstraction of fine-grained action identities using information such as tool names, documentation, and parameter schemas, potentially followed by text embeddings, clustering, centroid assignment, and approximate-nearest neighbor indexing.

The supplied README establishes the existence of the CES normalization and the semantic abstraction layer in the repository architecture, but it does not provide sufficient experimental evidence to assert a measured semantic-clustering F1, compression ratio, cluster purity, or downstream improvement.

Accordingly, this disclosure distinguishes the mechanism from its experimental validation. Where the implementation exposes semantic action representations, those representations are part of the implemented system. Claims of quantitative superiority of the semantic abstraction mechanism remain unsupported by the supplied benchmark results.

7.6 Protocol Inference

The implemented inference path constructs a Prefix Tree Acceptor from symbolic traces and uses state-merging mechanisms including EDMS and ALERGIA/MDI-style statistical compatibility.

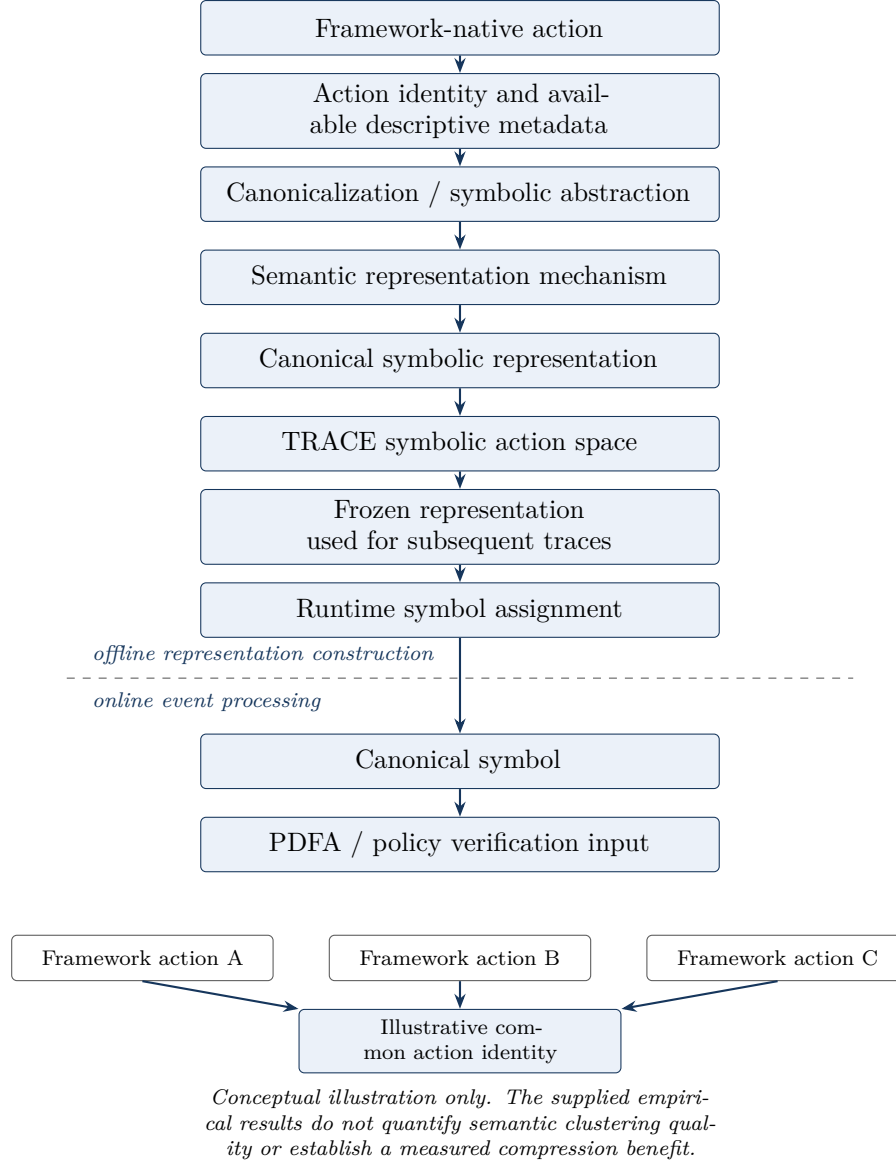


Figure 3: Semantic abstraction architecture. The system separates source-specific event representation from the canonical symbolic representation consumed by the inference and verification layers. Embedding, clustering, centroid assignment, and ANN mechanisms described in the architecture should not be interpreted as having a measured benchmark result unless such a result is explicitly supplied.

RPNI is not asserted as the principal TRACE inference mechanism and is not claimed as inventive. The primary implemented machinery is based on probabilistic state merging and EDSM-style inference.

The resulting representation is a probabilistic deterministic finite automaton (PDFA). No unsupported claim of Hopcroft-style minimization is made.

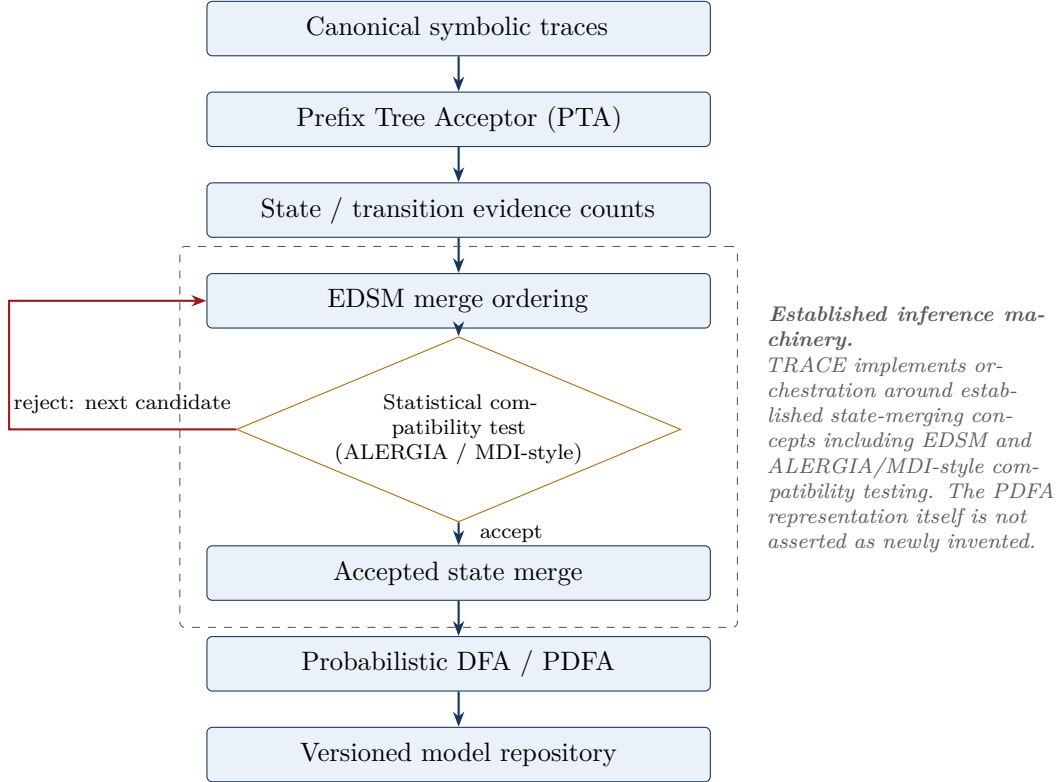


Figure 4: Implemented probabilistic automata inference pipeline. Canonical symbolic traces are converted into a PTA, candidate state merges are ordered using EDSM-style evidence, compatibility is evaluated using ALERGIA/MDI-style statistical criteria, and accepted merges produce a probabilistic finite-state behavioral model. The current disclosure does not assert Hopcroft-style minimization of the learned PDFA.

7.7 Hierarchical Trace Folding

The implementation represents multi-agent delegation using role-specific corpora and explicit delegation boundaries.

The documented implementation includes:

- `delegate(role)` boundary symbols;
- parent and child trace relationships;
- role-specific corpora;
- `parent_span_id` linkage;
- orphan-span / missing-child handling; and
- `MAX_DELEGATION_DEPTH = 8`.

The implementation therefore uses bounded hierarchical composition. It does not establish support for unbounded recursion or a general pushdown automaton.

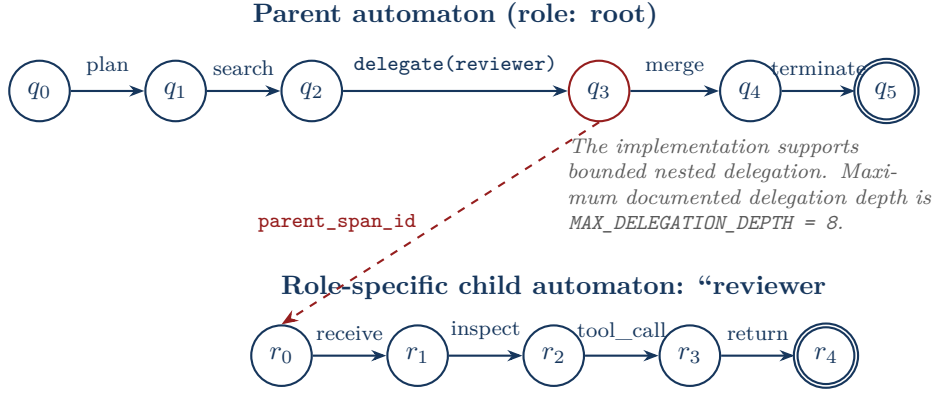


Figure 5: Hierarchical delegation representation. A parent trace retains a **delegate(role)** boundary while the delegated execution is represented using a role-specific child corpus and automaton linked through trace-span information. The implementation uses bounded-depth hierarchical composition and does not claim unbounded pushdown-automaton semantics.

Mathematically, the parent language remains regular over an alphabet containing **delegate(role)** as an atomic symbol. A role-specific child automaton is associated with the delegation boundary rather than expanded into the parent state machine.

This provides a bounded hierarchical representation while avoiding the claim that TRACE models arbitrary unbounded call-stack behavior.

7.8 Runtime Verification and the Product Automaton

The learned behavioral model is represented as

$$A = (Q_A, \Sigma, \delta_A, q_{A0}, P_A),$$

where Q_A is the learned state set, Σ is the canonical symbolic alphabet, δ_A is the learned transition function, q_{A0} is the initial state, and P_A represents transition probabilities.

The explicit policy DFA is represented as

$$P = (Q_P, \Sigma, \delta_P, q_{P0}, F_P),$$

where Q_P is the policy-state set, δ_P is the deterministic policy transition function, q_{P0} is the policy initial state, and F_P represents its accepting states.

The verifier tracks

$$(q_A, q_P) \in Q_A \times Q_P.$$

The learned and policy components are evaluated independently.

The primary classification precedence is:

1. **Policy violation:** the observed transition violates the explicit policy DFA.
2. **Structural anomaly:** the transition is not represented as an observed transition from the current learned state.
3. **Statistical anomaly:** the transition exists but its learned probability falls below the configured threshold ε .

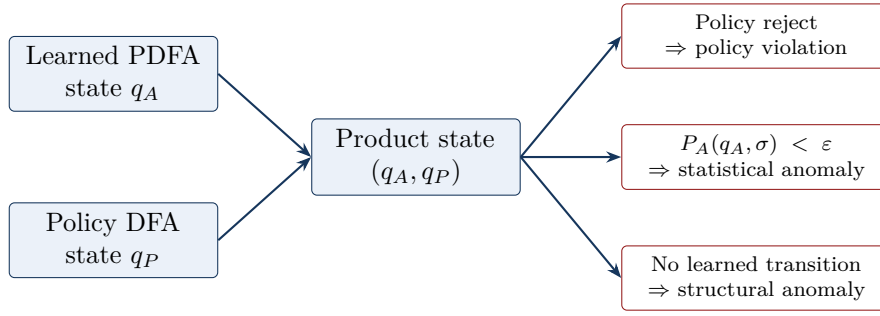


Figure 6: Product-state runtime verification. The learned probabilistic model and explicit policy DFA are evaluated independently while the verifier tracks their joint state. Explicit policy violations take precedence over learned statistical conditions; structural and statistical anomaly signals are retained as separate diagnostic classes.

4. **Conformant**: none of the above conditions applies.

The verifier may retain secondary flags when more than one condition is diagnostically relevant.

This is runtime verification of an observed execution. It is not exhaustive model checking of every possible execution path.

7.9 Counterexample Generation

The implemented explainability mechanism produces human-readable differences associated with detected deviations.

A violation explanation can identify the relevant unexpected transition and provide expected-versus-observed information rather than exposing only raw execution logs.

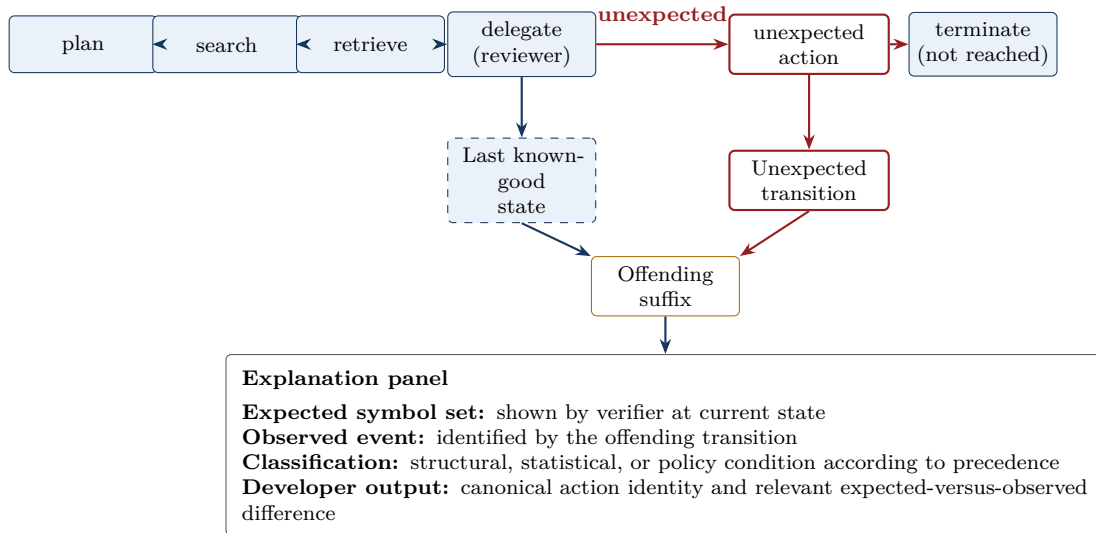


Figure 7: Conceptual counterexample explanation. The verifier identifies the relevant last-known-good state and the offending transition, after which the explainability layer produces an expected-versus-observed diagnostic. The example is illustrative and is not a benchmark result.

7.10 Drift Detection

The implementation includes behavioral drift monitoring based on rolling conformance statistics.

The supplied README documents two complementary mechanisms:

1. a two-sample Kolmogorov–Smirnov (KS) test over shifted trace distributions; and
2. tail-quantile CUSUM monitoring.

The system therefore does not treat KS as merely a future proposal. The current implementation includes the KS and tail-quantile CUSUM mechanisms.

The supplied empirical result for RQ5 is:

$$KS = 0.9800, \quad \text{triggered.}$$

This demonstrates the reported benchmark condition, but does not establish universal drift-detection accuracy, false-positive rate, detection latency, or performance across arbitrary non-stationary workloads.

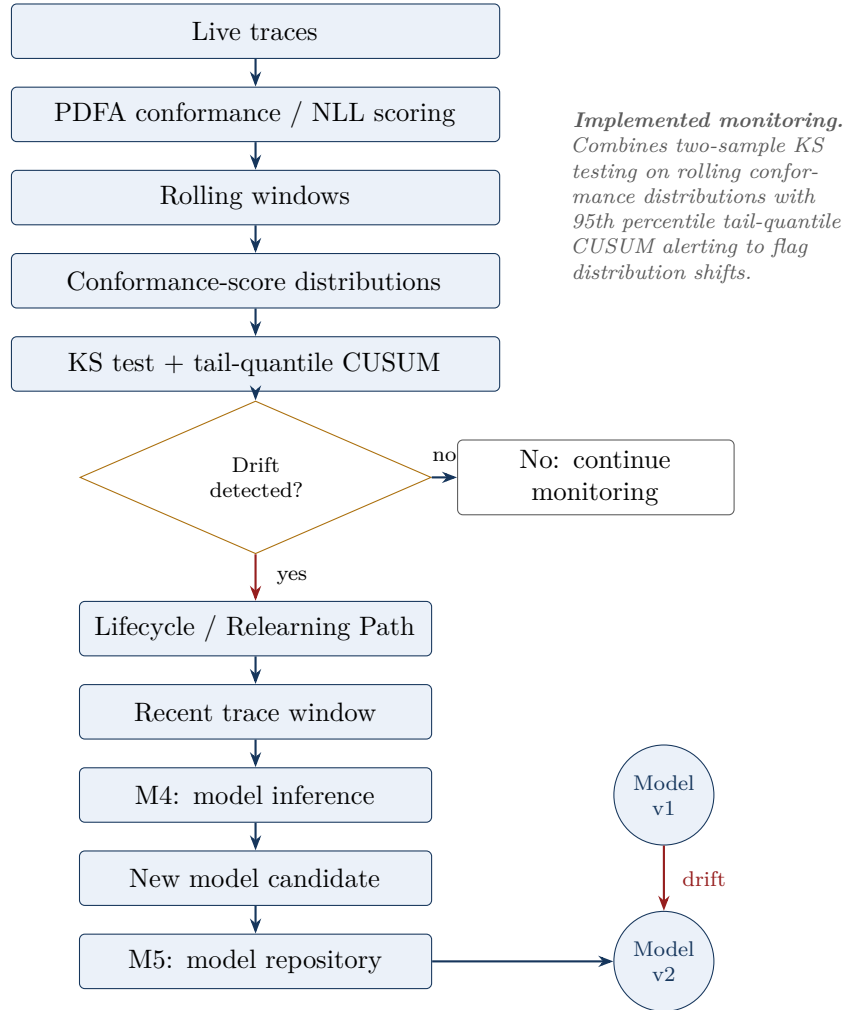


Figure 8: Behavioral drift monitoring. Rolling conformance distributions are evaluated using the implemented KS and tail-quantile CUSUM mechanisms. A detected shift can enter the model lifecycle/relearning path. The reported RQ5 benchmark demonstrates a triggered KS result but does not constitute a universal drift-detection guarantee.

7.11 Computational Complexity

7.12 Security and Failure Modes

Module	Complexity / Consideration
M1 Ingestion	$O(n)$ in trace length for per-event parsing, subject to source-specific parser costs.
M2 Semantic abstraction	Embedding and representation costs depend on corpus size and representation dimension. If nearest-centroid or ANN lookup is used, practical runtime depends on the selected index structure.
M4 State-merging inference	State merging can have substantial computational cost and does not receive a universal polynomial-time guarantee from the practical EDSM/ALERGIA-style search process.
M6 Runtime verification	Direct transition lookup can provide effectively constant-time state advancement per event for the active product state. A fully materialized product has worst-case state space $O(Q_A Q_P)$, while lazy evaluation can use less space.
M7 Drift monitoring	Window-based statistical monitoring depends on the selected window size and test implementation and is ordinarily evaluated periodically rather than requiring the full window computation on every event.

Table 5: Module-level computational considerations. The measured RQ3 latency is a verification-engine benchmark result and should not be interpreted as a statement that the entire end-to-end agent/tool/network interaction has the same latency.

Implemented Mitigations

- **Secret redaction:** entropy- and pattern-based recognition of documented secret classes before parameter-schema hashing.
- **Schema validation:** CES v1.0 JSON Schema draft 2020-12 validation.
- **Dead-letter handling:** malformed or invalid schema records can be separated from the valid processing path.
- **Policy validation:** explicit policy compilation and validation mechanisms.
- **Bounded delegation depth:** `MAX_DELEGATION_DEPTH = 8` limits hierarchical nesting in the current implementation.

Remaining Risks

- **Poisoned training traces:** an attacker able to inject malicious or misleading traces into model-building data could influence the learned behavioral model. The supplied implementation does not establish a complete training-data-poisoning defense.
- **Semantic over-clustering:** distinct actions may be mapped to insufficiently distinct symbolic representations, potentially masking a behavioral difference.
- **Semantic under-clustering:** excessively fragmented symbolic representations can increase alphabet size and complicate inference.
- **Adversarial tool naming:** malicious tools could attempt to exploit semantic similarity with trusted actions.
- **Model staleness:** drift monitoring provides a mechanism for identifying distribution changes, but the supplied evidence does not establish general robustness against all forms of model or workload evolution.

- **Incomplete traces:** truncated or missing events can affect learned state transitions and conformance interpretation.
- **Rare legitimate behavior:** legitimate but uncommon actions may be classified as statistically anomalous.
- **Policy conflicts:** contradictory or overly restrictive policy rules may produce undesired policy states and require policy validation.
- **Deployment-scale performance:** the reported latency benchmark does not establish production-scale performance under arbitrary concurrent agent workloads.
- **Adversarial evasion:** the current evidence does not establish robustness against an attacker deliberately optimizing behavior to remain inside the learned behavioral distribution.

7.13 End-to-End Operational Swimlane

Figure 9 separates the per-event runtime path from the feedback and model-lifecycle path.

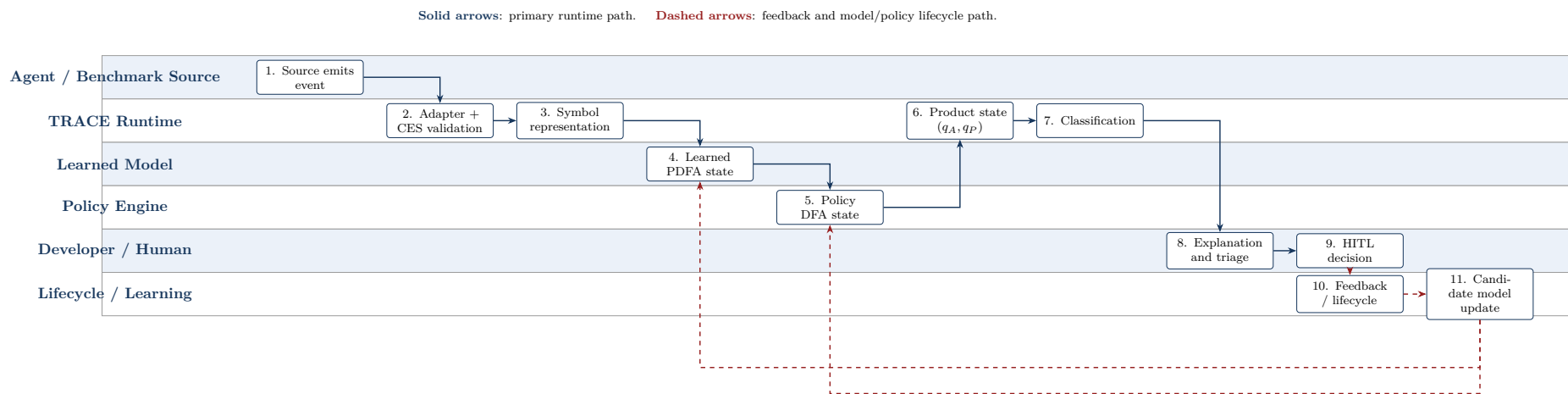


Figure 9: End-to-end operational workflow. A source event enters through an adapter and CES validation, is converted into the symbolic representation consumed by the learned and policy tracks, and is evaluated using the joint product state. Deviations can enter HITL triage and the model/policy lifecycle. The figure describes the implemented architecture and does not imply that every deployment automatically performs every lifecycle action.

7.14 Operational Interfaces and Deployment

The current implementation provides the following documented interfaces.

FastAPI

The FastAPI service exposes:

- `/v1/events` for event ingestion;
- `/v1/verify` for verification;
- `/dashboard` for the web console; and
- `/metrics` for Prometheus telemetry.

Dashboard

The dashboard provides an interactive interface for documented TRACE capabilities, including automata visualization, verification interaction, HITL triage, and benchmark presentation.

CLI and REPL

The implementation provides an interactive `trace` shell and commands for adapter inspection, policy validation, ingestion, model training, model inspection, validation, promotion, replay, verification, feedback, and benchmark execution.

The repository documents a 14-step demonstration accessible through:

```
trace demo
```

or non-interactively through:

```
trace demo --auto
```

Containerization

Docker and Docker Compose deployment are documented for the API service.

7.15 Recommended Patent Drawing Set

Figure	Content
Fig. 1	Overall TRACE implementation architecture and closed-loop pipeline.
Fig. 2	Seven framework adapters plus two benchmark trajectory adapters feeding CES v1.0.
Fig. 3	Semantic abstraction architecture, explicitly identified as an architectural mechanism without unsupported quantitative performance claims.
Fig. 4	PTA construction, EDSM ordering, ALERGIA/MDI-style compatibility, state merging, and PDFa repository.
Fig. 5	Bounded hierarchical delegation with role-specific child automata.
Fig. 6	Learned PDFa and explicit policy DFA tracked through a joint product state.
Fig. 7	Counterexample and expected-versus-observed explanation flow.
Fig. 8	Implemented KS and tail-quantile CUSUM drift monitoring and lifecycle path.
Fig. 9	End-to-end runtime and feedback/lifecycle workflow.

Table 6: Recommended figure set for formal patent-drawing preparation.

8. Experimental Validation Results

8.1 Current Implementation Status

The supplied repository README establishes that TRACE v2.0 is an implemented software prototype rather than a concept-only architecture.

The documented implementation includes:

- nine active trace adapters;
- CES v1.0 and JSON Schema draft 2020-12 validation;
- entropy- and pattern-based secret redaction;
- dead-letter handling;
- SQLite persistence;
- PTA construction;
- ALERGIA/MDI-style state merging;
- EDSM;
- PDFA representation;
- bounded hierarchical delegation;
- policy DSL and DFA compilation;
- product-automaton runtime verification;
- structural, statistical, and policy anomaly classification;
- running NLL scoring;
- counterexample explanation;
- KS and tail-quantile CUSUM drift monitoring;
- HITL triage and model lifecycle;
- FastAPI endpoints;
- dashboard;
- Prometheus telemetry;
- Docker and Docker Compose;
- interactive CLI and REPL;
- 14-step capability demonstration; and
- empirical benchmark execution infrastructure.

The repository additionally documents 57 automated software tests with a 100% pass rate.

The 57 automated tests should not be conflated with scientific validation of the research questions. Software tests establish that the tested implementation contracts and fixtures pass under the test suite. The empirical benchmark results described below address separate research questions.

8.2 Automated Testing Evidence

The documented automated test matrix is:

Test Suite	Target Layer	Tests	Result
Adapters	Seven core framework adapters	8	Passed
Benchmark	SWE-bench and OSWorld importers	5	Passed
CLI UI	Semantic CLI and REPL	9	Passed
CLI mock	Mock-data lifecycle	1	Passed
Golden	End-to-end pipeline fixture	1	Passed
Hierarchical	Multi-agent role isolation	7	Passed
Inference	PTA and ALERGIA state merging	4	Passed
Phase 2	TraceStore, quarantine, feedback	5	Passed
Phase 3	FastAPI, streaming, workers	5	Passed
Policy	Policy DSL and DFA compiler	4	Passed
Schema	CES and secret redaction	4	Passed
Verification	Dual-control verifier and inline guard	4	Passed
Total	Full automated suite	57	100%

Table 7: Automated software-test evidence documented by the TRACE repository. The 57 passing tests are implementation and contract evidence and should not be presented as proof of general scientific validity.

8.3 Empirical Research-Question Results

The supplied README contains numerical empirical results for **RQ1 through RQ5**. Although some repository labels refer to an “RQ1–RQ6” benchmark harness, no numerical RQ6 result is supplied in the provided material.

Accordingly, this disclosure reports only RQ1–RQ5 as demonstrated empirical results.

Research Question	Metric	Measured / Evaluated Result	Target	Status
RQ1: Sample Complexity	PDFA convergence on held-out NLL	$N = 50$ traces	≤ 100 traces	Passed
RQ2: Automata Recovery	Precision / Recall / F1	$F_1 = 0.9615$; Precision = 0.9259; Recall = 1.0000	$F_1 \geq 0.90$	Passed
RQ3: Verification Latency	Single-event verification latency	$p99 = 0.0206$ ms; $p50 = 0.015$ ms	≤ 5.0 ms	Passed
RQ4: Policy Enforcement	Safety-violation detection accuracy	100.0% accuracy; 0 false negatives	100% detection	Passed
RQ5: Drift Detection	Two-sample KS test on shifted traces	KS = 0.9800; triggered	Flag shift	Passed
RQ6: State Space Reduction	Hierarchical vs. flat state count (analytical estimate)	$48 \rightarrow 14$ states (70.83% reduction; $ Q_{\text{parent}} = 8, Q_{\text{child}} = 6$)	State space compression	Analytical estimate
Semantic Abstraction Ablation	Multi-adapter vocabulary compression	$28 \rightarrow 10$ symbols (64.29% compression across adapters)	Alphabet reduction	Passed

Table 8: Benchmark evaluation results for TRACE v2.0. RQ1–RQ5 are empirical measurements from the benchmark evaluation harness. RQ6 is an analytical state-space estimate illustrating the mathematical difference between flat and hierarchical representation under fixed illustrative parameters; it is not an empirical measurement from a real agent workload.

8.4 RQ1: Sample Complexity

The supplied benchmark reports convergence of the PDFA evaluation at

$$N = 50$$

traces against a target of no more than 100 traces.

This is a measured result for the reported experimental configuration. It should not be interpreted as a general sample-complexity theorem for all TRACE models, alphabets, workloads, or stochastic processes.

8.5 RQ2: Automata Recovery

The supplied benchmark reports:

$$F_1 = 0.9615, \quad \text{Precision} = 0.9259, \quad \text{Recall} = 1.0000.$$

The reported target was $F_1 \geq 0.90$.

These values constitute reported benchmark measurements and not a universal accuracy guarantee for TRACE.

8.6 RQ3: Verification Latency

The supplied benchmark reports:

$$p99 = 0.0206 \text{ ms}, \quad p50 = 0.015 \text{ ms}.$$

Against the stated 5.0 ms budget,

$$\frac{5.0}{0.0206} \approx 242.7.$$

Thus the measured p99 verification latency is approximately **243 times below the stated latency budget** under the reported evaluation conditions.

This result refers to the TRACE verification engine. It does not mean that the entire AI-agent interaction, tool execution, network communication, model inference, or end-to-end request takes 0.0206 ms.

The result is a benchmark-specific measurement rather than a universal performance guarantee.

8.7 RQ4: Policy Enforcement

The supplied benchmark reports:

$$100.0\% \text{ accuracy}, \quad 0 \text{ false negatives}.$$

This result meets the stated 100% detection target on the reported benchmark.

The result must not be interpreted as a universal guarantee of zero false negatives for arbitrary policies, workloads, adversarial inputs, or deployments.

8.8 RQ5: Drift Detection

The supplied benchmark reports:

$$KS = 0.9800, \quad \text{triggered.}$$

The implementation also includes tail-quantile CUSUM monitoring.

The reported KS result demonstrates detection for the supplied shifted-trace benchmark. The supplied material does not provide sufficient measurements of detection latency, false-positive rate, false-negative rate, or robustness across arbitrary non-stationary workloads.

8.9 RQ6 Status: Hierarchical State Space Reduction (Analytical Estimate)

The benchmark evaluation harness includes an RQ6 module evaluating hierarchical state space reduction. As documented in the repository benchmark harness (`trace.benchmarks.evaluator`), this evaluation provides an **analytical estimate** illustrating the theoretical state-space difference between a flat product automaton and a bounded hierarchical composition, rather than an empirical measurement from a live agent workload:

- **Mathematical model:** For a parent workflow with state set Q_{parent} delegating to a child workflow with state set Q_{child} , a flat composition requires $|Q_{\text{parent}}| \cdot |Q_{\text{child}}|$ states.
- **Hierarchical composition:** Under TRACE bounded hierarchical composition, the parent and child automata remain decoupled and linked via delegation boundaries and `parent_span_id`, requiring only $|Q_{\text{parent}}| + |Q_{\text{child}}|$ states.
- **Illustrative example:** For example sizes $|Q_{\text{parent}}| = 8$ and $|Q_{\text{child}}| = 6$:

$$\text{Flat states} = 8 \times 6 = 48, \quad \text{Hierarchical states} = 8 + 6 = 14,$$

yielding an analytical state-space reduction of:

$$\frac{48 - 14}{48} = \frac{34}{48} \approx \mathbf{70.83\%}.$$

Qualification: Empirical benchmark measurements are claimed strictly for RQ1–RQ5. The RQ6 reduction percentage (70.83%) represents an analytical mathematical estimate on fixed illustrative parameters and must not be cited as an empirical measurement from a production or multi-agent execution workload.

8.10 Semantic Abstraction Ablation

The benchmark evaluation suite measures the vocabulary compression achieved by semantic canonicalization across the supported framework adapters (AutoGen, CrewAI, Google ADK, LangGraph, Model Context Protocol, OpenAI Agents SDK, OSWorld, Semantic Kernel, and SWE-bench):

$$\begin{aligned} \text{Raw Vocabulary} &= 28 \text{ framework action signatures,} \\ \text{Canonical Alphabet } (\Sigma) &= 10 \text{ symbols,} \\ \text{Vocabulary Compression} &= \mathbf{64.29\%}. \end{aligned}$$

This empirical ablation demonstrates that canonicalizing heterogeneous framework actions into the standardized alphabet Σ achieves a 64.29% reduction in the active alphabet size, directly mitigating state-space explosion in the prefix tree acceptor (PTA).

8.11 Interpretation of the Current Evidence

The current evidence establishes three distinct levels of validation:

1. **Implementation evidence:** the repository documents working modules, interfaces, adapters, deployment mechanisms, and lifecycle functionality.
2. **Software-test evidence:** 57 automated tests are documented as passing.
3. **Empirical benchmark evidence:** RQ1–RQ5 and semantic ablation have reported measurements.

These categories should remain separate. In particular:

- 57 passing tests are not equivalent to scientific proof;
- $F_1 = 0.9615$ is not a universal recovery guarantee;
- 100% policy accuracy is not a universal zero-false-negative guarantee;
- 0.0206 ms p99 is not total agent latency;
- $KS = 0.9800$ is not universal drift-detection performance; and
- $N = 50$ is not a universal sample-complexity theorem.

8.12 Evidence Not Currently Supplied

While vocabulary compression (64.29%) and analytical state-space reduction (70.83%) are documented, the supplied evaluation does not establish:

- semantic abstraction cluster purity, F1, or Silhouette scores;
- empirical multi-agent workload traces for RQ6 (beyond the analytical formula);
- drift false-positive and false-negative rate curves across shifting agent prompts;
- drift detection latency under streaming loads;
- deployment-scale operational concurrency limits;
- in-distribution adversarial evasion robustness; or
- formal verification of unbounded recursive delegation.

These remain appropriate future evaluation areas.

9. Aspects of the Invention Needing Protection

The following identifies candidate technical claim scope for discussion with patent counsel. It is not a representation that any identified element or combination is legally patentable, novel, non-obvious, valid, or enforceable.

9.1 Potentially Distinguishing System Architecture

The candidate system-level combination is:

heterogeneous trace ingestion → canonicalization
 → symbol abstraction
 → probabilistic state-merging inference
 → bounded delegation representation
 → learned/policy product verification
 → drift monitoring
 → HITL/model lifecycle.

The potential technical contribution lies in the interaction between these mechanisms when applied to heterogeneous AI-agent execution traces, rather than in any one algorithm in isolation.

9.2 Potentially Distinguishing Method

A candidate method may comprise:

1. receiving execution-trace events from multiple structurally distinct agent execution frameworks and/or benchmark trajectory sources;
2. mapping received events into a common canonical representation;
3. applying recognized secret redaction before parameter-schema hashing;
4. constructing symbolic traces for behavioral inference;
5. constructing a Prefix Tree Acceptor from the symbolic traces;
6. applying EDSM ordering and ALERGIA/MDI-style statistical state-merging to obtain a probabilistic behavioral model;
7. upon detecting a delegation boundary, associating the delegated trace with a role-specific corpus and bounded child automaton;
8. maintaining an explicit policy DFA independently from the learned behavioral model;
9. tracking the learned and policy states jointly as $(q_A, q_P) \in Q_A \times Q_P$;
10. evaluating explicit policy conformance independently of learned statistical conformance;
11. classifying deviations using explicit policy, structural, and statistical conditions;
12. generating developer-facing counterexample information;
13. monitoring rolling conformance distributions using KS and tail-quantile CUSUM mechanisms; and
14. incorporating HITL feedback into model and policy lifecycle operations.

9.3 Potentially Distinguishing Data Representation

The CES representation acts as a cross-framework join point between source-specific execution events and downstream TRACE components.

The candidate representation includes source identity, event categorization, symbolic action information, attributes, timestamps, span relationships, and delegation depth.

Its treatment as a claimable data structure per se requires separate patent-counsel analysis. The stronger technical focus may instead be the processing and verification method in which the representation is used.

9.4 Potentially Distinguishing Anomaly-Detection Mechanism

A candidate technical distinction is the retention of separate semantic outcomes for:

- (a) structurally unseen transitions;
- (b) statistically low-probability transitions; and
- (c) explicit policy violations.

The implementation distinguishes these conditions rather than treating all deviations as a single undifferentiated anomaly score.

The primary outcome precedence is:

policy violation > structural anomaly > statistical anomaly > conformant.

Secondary diagnostic flags may be retained where multiple conditions are simultaneously present.

9.5 Potentially Distinguishing Hierarchical Mechanism

The implemented hierarchical mechanism uses:

- parent traces;
- `delegate(role)` boundary symbols;
- role-specific child corpora;
- `parent_span_id` relationships; and
- a documented maximum delegation depth of 8.

This should be described as bounded-depth hierarchical composition, not as an unbounded push-down automaton.

9.6 Potentially Distinguishing Drift / Lifecycle Mechanism

The implemented system combines behavioral conformance monitoring with:

- rolling distributions;
- two-sample KS testing;
- tail-quantile CUSUM monitoring; and
- model lifecycle operations.

The current benchmark demonstrates the reported KS drift result but does not establish universal drift-detection performance.

9.7 Potentially Distinguishing Feedback Mechanism

The implemented HITL mechanism provides violation triage and candidate model lifecycle operations.

The candidate system-level distinction is the integration of operator feedback with a behavioral-model lifecycle in which candidate models can transition through:

CANDIDATE → SHADOWED → ACTIVE → SUPERSEDED.

Any claim concerning automatic adjustment of particular statistical parameters should be limited to behavior actually supported by the implementation evidence.

9.8 Independent Claim Concept

Concept only — not proposed as filed claim language.

“A computer-implemented method for monitoring execution behavior of an AI-agent system, the method comprising: receiving execution-trace events from multiple structurally distinct agent execution sources; converting the execution-trace events into a common canonical representation; constructing symbolic execution traces from the canonical representation; constructing a probabilistic finite-state behavioral model from the symbolic execution traces using state-merging inference; representing a delegated execution as a bounded-depth role-associated sub-trace; receiving a live execution event and advancing a state of the probabilistic behavioral model and a state of a separately specified policy automaton; tracking the two states as a joint verification state; determining independently whether the event violates the policy automaton, is structurally unrepresented by the probabilistic behavioral model, or has a transition probability below a configured threshold; and generating an explanation identifying the detected deviation.”

9.9 Possible Dependent-Claim Concepts

- (a) wherein the canonical representation includes source-independent event categorization and a fine-grained symbolic action identity;
- (b) wherein recognized secrets are removed using entropy- and pattern-based redaction before parameter-schema hashing;
- (c) wherein the probabilistic behavioral model is generated using EDSM and ALERGIA/MDI-style statistical compatibility;
- (d) wherein a delegated trace is associated with a role-specific corpus and a maximum delegation depth;
- (e) wherein the runtime verifier maintains a joint state (q_A, q_P) ;
- (f) wherein explicit policy violations have precedence over statistical anomaly classification;
- (g) wherein drift is monitored using a two-sample KS test and tail-quantile CUSUM;
- (h) wherein a detected deviation can enter a candidate model lifecycle;
- (i) wherein the system provides human-in-the-loop triage; and
- (j) wherein the system exposes runtime verification through an API, dashboard, or command-line interface.

9.10 Novelty versus Obviousness

Established components. The following are established technologies or general technical concepts and should not be treated as individually newly invented by TRACE:

- RPNI;
- EDSM;
- ALERGIA/MDI;
- FlexFringe;
- probabilistic finite automata;
- embeddings;
- clustering;

- runtime verification;
- product automata;
- process mining;
- anomaly detection;
- hierarchical software-trace modeling; and
- human-in-the-loop review as a general design pattern.

Candidate system-level distinction. The supplied prior-art review did not identify a single reference disclosing the complete combination of cross-framework agent-trace normalization, semantic action representation, probabilistic state-merging inference, bounded hierarchical delegation, independent explicit policy verification through a product state, structural/statistical classification, drift monitoring, and HITL/model lifecycle management.

This is a statement about the supplied technical review, not a legal conclusion about novelty.

Inventive-step issue. The strongest question for formal patent counsel is whether the particular combination and technical interactions would have been obvious in view of the identified prior art, including TraceAegis and established automata-learning and runtime-verification techniques.

Technical interactions worth documenting. Particular attention should be given to:

1. the interaction between canonical cross-framework representation and probabilistic state inference;
2. the separate semantics of structural, statistical, and policy deviations;
3. bounded delegation and role-specific behavioral modeling;
4. independent policy enforcement that cannot be overridden by learned statistical normality;
5. the interaction between drift monitoring and model lifecycle; and
6. the practical behavior of the system under incomplete, rare, or adversarial traces.

10. Technology Readiness Level (TRL)

Recommended TRL: TRL 4 — Technology validated in a laboratory / research environment

Category: Development / Research Prototype

The current self-assessment is approximately TRL 4. This assessment reflects an implemented software system with controlled testing and benchmark validation. It does not imply production-scale deployment, industrial qualification, or operational-environment demonstration.

Evidence Supporting This Level

The supplied repository documents:

- an implemented TRACE software package;
- nine active adapters;
- CES v1.0 validation and secret redaction;
- implemented automata inference;
- hierarchical delegation support;

Category	TRL	Description	Status
Research	TRL 1	Basic principles observed	☐
	TRL 2	Technology concept formulated	☐
	TRL 3	Experimental proof of concept	☐
Development	TRL 4	Technology validated in a laboratory / research environment	☒
	TRL 5	Technology validated in a relevant environment	☐
	TRL 6	Technology demonstrated in a relevant environment	☐
Deployment	TRL 7	System prototype demonstration in an operational environment	☐
	TRL 8	System complete and qualified	☐
	TRL 9	Actual system proven in an operational environment	☐

Table 9: Technology Readiness Level self-assessment. TRL 4 is selected based on the implemented prototype, controlled automated testing, and reported empirical benchmark results.

- policy compilation;
- runtime product verification;
- drift monitoring;
- HITL feedback;
- FastAPI service endpoints;
- dashboard;
- Prometheus telemetry;
- Docker and Docker Compose;
- CLI and REPL;
- a 14-step demonstration;
- 57 automated tests with 100% passing results; and
- empirical benchmark results for RQ1–RQ5.

These elements support a laboratory/research-environment validation assessment beyond concept-only or proof-of-concept status.

Why TRL 5 or Higher Is Not Claimed

The supplied evidence does not establish:

- broad production deployment;
- validation under operational-scale concurrency;
- deployment across a representative range of real-world agent systems;
- adversarial robustness at operational scale;

- production reliability or service-level guarantees; or
- qualification in a relevant operational environment.

Accordingly, TRL 5–9 is not asserted.

Concrete Work Needed for Further Maturity

Future work may include:

1. broader validation using independently generated traces from additional real-world agent workloads;
2. larger training corpora and longer-running trajectories;
3. stronger comparisons against n-gram, Markov, process-mining, and other behavioral baselines;
4. continuous vector embedding clustering analysis and Silhouette metric evaluation;
5. explicit evaluation of drift false positives, false negatives, and detection latency;
6. deployment-scale concurrency and resource testing;
7. adversarial robustness testing;
8. advanced in-distribution adversarial training-data-poisoning defenses;
9. evaluation of incomplete and corrupted traces;
10. formal patent novelty and inventive-step review.

Executive Novelty Assessment

A. Current implementation status.

TRACE v2.0 is no longer a concept-only architecture. The supplied repository documents an implemented research-grade prototype with nine active trace adapters, CES validation, state-merging inference, bounded hierarchical delegation, policy compilation, product-state verification, drift monitoring, HITL functionality, API/dashboard interfaces, telemetry, containerization, and CLI tooling.

B. Software validation.

The repository documents 57 automated tests with a 100% pass rate. These tests provide implementation and contract evidence but should not be treated as scientific proof.

C. Empirical validation.

The supplied benchmark reports results for RQ1–RQ5 alongside a semantic abstraction ablation: $N = 50$ convergence for RQ1; $F_1 = 0.9615$ for RQ2; $p99 = 0.0206$ ms for RQ3; 100.0% benchmark policy accuracy for RQ4; $KS = 0.9800$, triggered, for RQ5; and a 64.29% vocabulary compression ablation ($28 \rightarrow 10$ symbols across adapters). RQ6 is documented as an analytical state-space reduction model (70.83%, $48 \rightarrow 14$ states); no empirical multi-agent runtime workload is claimed for RQ6.

D. Candidate technical distinction.

The candidate contribution is the system-level combination of heterogeneous trace canonicalization, semantic symbolic representation, probabilistic behavioral inference, bounded delegation modeling, independent policy verification through a product state, structural/statistical/policy classification, drift monitoring, and HITL/model lifecycle.

E. Established components.

RPNI, EDSM, ALERGIA/MDI, FlexFringe, probabilistic automata, embeddings, clustering, runtime verification, product automata, process mining, anomaly detection, and hierarchical

trace modeling are established techniques and should not themselves be presented as newly invented by TRACE.

F. Strongest patentability question.

The principal legal question is whether the particular combination and technical interactions would have been obvious in view of the identified prior art. TraceAegis is the closest identified contemporaneous reference in the supplied review, while established automata-learning and runtime-verification literature supplies additional potential combinations.

G. Remaining evidence gap.

The current supplied evidence does not establish continuous embedding cluster purity, empirical multi-agent execution workloads for RQ6, broad production-scale concurrency, adversarial evasion robustness, or general drift-detection operating characteristics.

H. Overall assessment.

The implemented TRACE v2.0 system has progressed materially beyond concept-only status. The strongest candidate contribution is the particular system-level combination and its technical interactions rather than any individual underlying algorithm. A formal patentability assessment should therefore focus on the precise combination, its technical effects and interactions, and the prior-art combinations available to a person skilled in the relevant art.

I. Counsel review.

Formal patent counsel review remains necessary before relying on this document for filing, including review of novelty, inventive step, enablement, claim scope, disclosure timing, and freedom to operate.

Critical Issues Requiring Inventor Review

1. **RQ6 is documented as an analytical estimate, not an empirical workload.** The benchmark harness and report document an analytical state-space reduction model for RQ6 (70.83% reduction, $|Q_P| \cdot |Q_C| = 48 \rightarrow |Q_P| + |Q_C| = 14$ states). This must remain explicitly characterized as a mathematical formula over illustrative parameters rather than an empirical measurement from a multi-agent production workload.
2. **Semantic abstraction vocabulary compression is documented; clustering purity remains open.** The benchmark evaluation documents an empirical 64.29% vocabulary compression ablation across the nine supported framework adapters ($28 \rightarrow 10$ symbols). However, continuous vector embedding clustering purity, F1, and Silhouette scores are not yet quantitatively evaluated and must remain qualified.
3. **Training-data poisoning defenses require continuous hardening.** The implementation now includes trace-level outlier filtering (`write_trace_safely`) and quarantine isolation (`quarantined_traces`) based on baseline PDFa log-likelihood. However, subtle in-distribution adversarial trace injection could still evade statistical rejection thresholds and requires defense-in-depth and human sign-off.
4. **Broader real-framework validation remains desirable.** The repository documents nine active framework adapters (AutoGen, CrewAI, Google ADK, LangGraph, Model Context Protocol, OpenAI Agents SDK, OSWorld, Semantic Kernel, and SWE-bench), but broader third-party independent workload validation would strengthen evidence concerning generality.
5. **Benchmark results are environment and dataset specific.** The reported RQ1–RQ5 values should not be generalized to arbitrary hardware, workloads, datasets, agent frameworks, or deployment environments.

6. **Production-scale validation is not established.** The implementation includes Docker, FastAPI, dashboard, telemetry, and streaming interfaces, but these facts do not establish operational-scale qualification.
7. **Adversarial robustness remains an open issue.** Future evaluation should consider adversarially selected tool names, deliberately manipulated trajectories, model evasion, and poisoned traces.
8. **TraceAegis requires careful attorney review.** It is the closest identified prior work in the supplied technical review. The relevant question is the precise technical difference and whether the claimed combination would have been obvious, not whether TRACE is categorically novel.
9. **Bounded recursion must remain explicit.** The current implementation documents `MAX_DELEGATION_DEPTH = 8`. The disclosure should not imply support for unbounded recursive execution.
10. **Policy and learned behavior must remain distinct.** The learned PDFA represents statistical behavioral conformance, while the policy DFA represents explicit constraints. Statistical normality must not be described as overriding explicit policy enforcement.
11. **The RQ3 latency number must remain scoped.** The reported $p99 = 0.0206$ ms applies to the measured verification engine benchmark. It must not be presented as end-to-end AI-agent, model-inference, tool, or network latency.
12. **The RQ4 result must remain benchmark-specific.** The reported 100.0% accuracy and zero false negatives are measurements on the supplied benchmark, not a universal guarantee.
13. **The RQ5 result must remain benchmark-specific.** $KS = 0.9800$ and a triggered result establish the reported benchmark outcome, not universal drift-detection performance.
14. **Formal patent search remains necessary.** The supplied prior-art analysis is a technical aid and should not substitute for a formal attorney-conducted novelty and inventive-step search.
15. **Patentability is not guaranteed.** The implementation evidence demonstrates a functioning research prototype and reported benchmark results, but those facts alone do not establish legal patentability.